

Test la Programarea Calculatoarelor - Subiect C

10. ian. 2022

Fiecare problemă va fi rezolvată într-un proiect separat. Fișierele sursă *main.c* de la fiecare proiect în parte se redenumesc *Nume_Prenume_prb1.c* și *Nume_Prenume_prb2.c*, apoi se încarcă pe Teams. Nu se punctează cod scris în comentariu.

1p – codul compilează, este încărcat cu nume corect, este formatat cu AStyle (0.5p per problema)

1p – rezultate corecte pe exemplele date în cerință (0.5p per problema)

1p – verificări și teste scrise pentru alte cazuri în comentarii (0.5p per problema)

Problema 1 (3p) - Problema 1

Un lanț de numere este format pornind de la un număr inițial și, adunând patratele cifrelor numărului, obținem un nou număr. Acest lanț se termină în momentul în care obținem un număr pe care l-am văzut în trecut.

De exemplu, pornind de la numărul 32 vom avea lanțul următor:

32 -> 13 -> 10 -> 1 -> 1.

Explicație : $32 \rightarrow 13 = 3^2 + 2^2 \rightarrow 10 = 1^2 + 3^2 \rightarrow 1 = 1^2 + 0^2 \rightarrow 1$

Acest lanț are lungimea 5.

Un alt exemplu, pornind de la numărul 85, obținem lanțul următor:

85 -> 89 -> 145 -> 42 -> 20 -> 4 -> 16 -> 37 -> 58 -> 89

Numărul 89 a fost văzut în trecut, deci ne oprim. Acest lanț are lungimea 10.

a) Să se citească de la tastatură un număr întreg $n < 10^9$ și două numere naturale m și k astfel încât $m \cdot k < 10^8$ și să le salveze.

b) Să se scrie o funcție care primește ca parametrii numărul citit n și un tablou v alocat în funcția părinte și care salvează în vectorul v lanțul pentru numărul n și returnează lungimea acestuia. Să se afișeze lanțul returnat după forma de mai sus.

Exemplu pentru $n = 12$:

5 -> 25 -> 29 -> 85 -> **89** -> 145 -> 42 -> 20 -> 4 -> 16 -> 37 -> 58 -> **89** (89 apare de 2 ori)

c) Să se scrie o funcție care primește ca parametru numărul m și numărul k . Să se determine toate numerele mai mici ca m a căror lanțuri au lungimea mai mică sau egală cu k .

Exemplu: pentru $m = 5$ și $k = 10$, lanțurile care îndeplinesc condiția sunt:

Pentru 1 : 1 -> 1

Pentru 2 : 2 -> 4 -> 16 -> 37 -> 58 -> 89 -> 145 -> 42 -> 20 -> 4

Pentru 4 : 4 -> 16 -> 37 -> 58 -> 89 -> 145 -> 42 -> 20 -> 4 -> 16

Problema 2 (4p) - Problema 2

Se consideră fișierul *cuvinte.txt* mai mic de 1 MB care conține un număr necunoscut de cuvinte separate prin spațiu. Cuvintele conțin doar litere mici.

a) Să se citească cuvintele din fișier și să se salveze în memorie folosind alocare dinamică.

b) Să se scrie o funcție recursivă care determină dacă un șir de caractere este sau nu palindrom. Pentru punctaj maxim funcția va avea 2 pointeri ca parametri de intrare.

Exemplu: "abba" este palindrom, "ba" nu este palindrom.

c) Să se scrie o funcție care determină câte perechi de cuvinte din cele citite la punctul a) există ale căror concatenare este un șir palindrom.

Exemplu: dacă în fișier se află cuvintele "abc", "cba", "tas", "sat", "abc" se va returna 2, perechiile fiind ("abc", "cba") și ("tas", "sat").

d) Pentru cuvintele citite la punctul a) care nu sunt palindroame, să se determine cel mai lung subșir al cuvântului care este palindrom. Să se scrie subșirul într-un alt fișier numit *subiruri.txt* care conține pe fiecare linie cuvântul urmat de subșirul găsit.

Exemplu: pentru cuvântul "abcbbbbcb", în fișier va fi scris: "abcbbbbcb cbbbbbcb"

pentru cuvântul "abxcabcabaabxx", în fișier va fi scris: "abxcabcabbaabxx abbba"